

LAB 1: SMART THERMOMETER

JOSEPH KRUEGER, CAVAN RILEY, HOLLAND GILMORE, AND BRI
VILLARREAL

A REPORT SUBMITTED AS PART OF THE REQUIREMENTS FOR LAB 1

4 October 2024

ECE:4880
Principles of ECE/CSE Design
Team 15

Contents

1	Design Documentation	1
1.1	Hardware Design: Circuitry	1
1.1.1	Requirements	1
1.1.2	Overall System	1
1.2	Hardware Design: Schematics	2
1.3	Software Design: ESP32 Firmware	3
1.3.1	ESP32	3
1.3.2	Main Control Loop	3
1.3.3	Interrupts	4
1.4	Software Design: ESP32 Web Server	4
1.4.1	Setup	4
1.4.2	HTTP Interactions	4
1.5	Software Design: GUI	5
1.5.1	Routes Interacting With Server	5
1.5.2	Routes Without Server Interaction	5
1.6	Software Design: Schematics	6
2	Design Process and Experimentation	8
2.1	Hardware Selection: "Third Box" Circuitry	8
2.1.1	Hardware Constraints	8
2.1.2	Voltage Differences	9
2.1.3	Power Requirements	10
2.2	Hardware Selection: "Third Box" Construction	11
2.3	Software Selection: ESP32 Firmware	12
2.4	Software Selection: ESP32 Web Server	12
2.4.1	Enterprise Wi-Fi Connection (eduroam)	12
2.4.2	Client-Server Communication	13
2.4.3	Final Design	14
2.5	Software Selection: GUI	14
2.5.1	Initial Thinking	14
2.5.2	Implementation	14
2.5.3	User Messaging	15

3	Test Report	16
3.1	Test Strategy	16
3.2	Hardware Tests	17
3.3	Software Tests	18
4	Project Retrospective	20
4.1	Project Management	20
4.2	Gantt Chart	20
4.3	Outcome Summary	21
4.3.1	Hardware	21
4.3.2	Software	21
4.4	Possible Improvements	22
4.4.1	Hardware	22
4.4.2	Server	22
4.4.3	GUI	22
4.5	Member Contributions	22
4.5.1	Cavan	23
4.5.2	Joseph	23
4.5.3	Bri	23
4.5.4	Holland	23
5	Appendix	24
5.1	Figures	24
5.2	Code	27
5.3	References	29

Chapter 1

Design Documentation

This section describes the relevant software and hardware design of the components of this project and how they interact with each other.

1.1 Hardware Design: Circuitry

1.1.1 Requirements

The system's hardware is divided into two parts: the wiring and the box. When building the circuit, we had requirements in mind for basic functionality. Ideally, the ESP32 should be able to read data from the temperature sensors and buttons simultaneously. When the buttons are pressed for the first time, the temperature data should be displayed on the LCD, such as 'Sensor 1: 22 C' on line 1. If a button is pressed again, the LCD should stop displaying the temperature data on the respective line. This was achieved through basic programming and our initial hardware design, which excluded a switch and battery. The initial hardware design, powered by a wall socket adaptor, made it possible to start the software development side of the system.

1.1.2 Overall System

The box design began by adding a 9V battery and a switch to the system. The parts were categorized into two groups: The user interface and the hidden wiring. The system diagram (see Figure [1.1]) includes all system components. When the system is turned on via the switch, a 9V battery powers the HW-131, which manages the differing voltage requirements of the LCM 1602A and the ESP32. When the push buttons are pressed, the ESP32 reads the data from the DS1820 temperature sensors. That data is then configured and sent to the LCD to display the temperature on the box. The added software enabled data transmission to an external computer over Wi-Fi, where it is simultaneously displayed on a graph. If the data being read is outside of the range decided by the user on the computer interface, a message/email is sent to a phone stating whether the data exceeded the maximum or fell below the minimum.

The user interface on the box includes the push buttons, the switch, the temperature sensor

cables, and the LCD (see Figure [5.1]). From this image, it is clear that the LCD's backlight intensity is set sufficiently to see through the plastic lid in indoor lighting. The buttons and switch are made accessible via the top of the box while the temperature sensor cables are accessible through the side of the box to measure temperature. The user controls those parts; they do not have access to the wiring that makes it possible to read and display the data. Everything inside the box (see Figure [5.2]) is what provides the functionality of the system. These components are hidden to maintain a simple and clean box design. For a detailed schematic of the circuitry within the box see Figure [1.2].

Keeping the design simple, the connectors are purposefully user-friendly. To disconnect a temperature sensor cable from the box (see Figure [5.3]), twist the handle to the left until you are able to pull the cable off of the connector. To reconnect, align the triangles as they are in the figure and plug them into the connector.

1.2 Hardware Design: Schematics

Below is the high-level system design, along with the previous wiring diagram, parts list, and key. Note that the computer and cell phone do not have a wired connection to the system, but are connected via Wi-Fi.

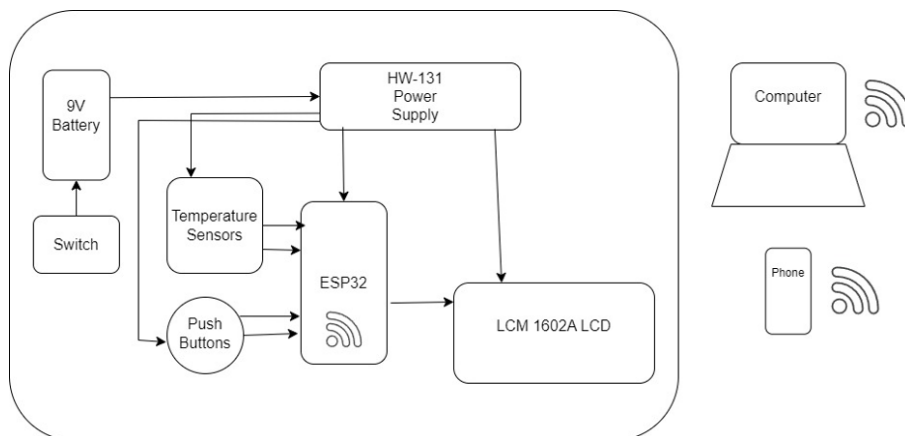


Figure 1.1: System Hardware Design Schematic

Below is the wiring pinout for the Smart Thermometer system. As you can see, the ESP32 interfaces with an LCD, 2 temperature sensors, and 2 push buttons. We constructed this system on a single breadboard, powered by the HW-131.

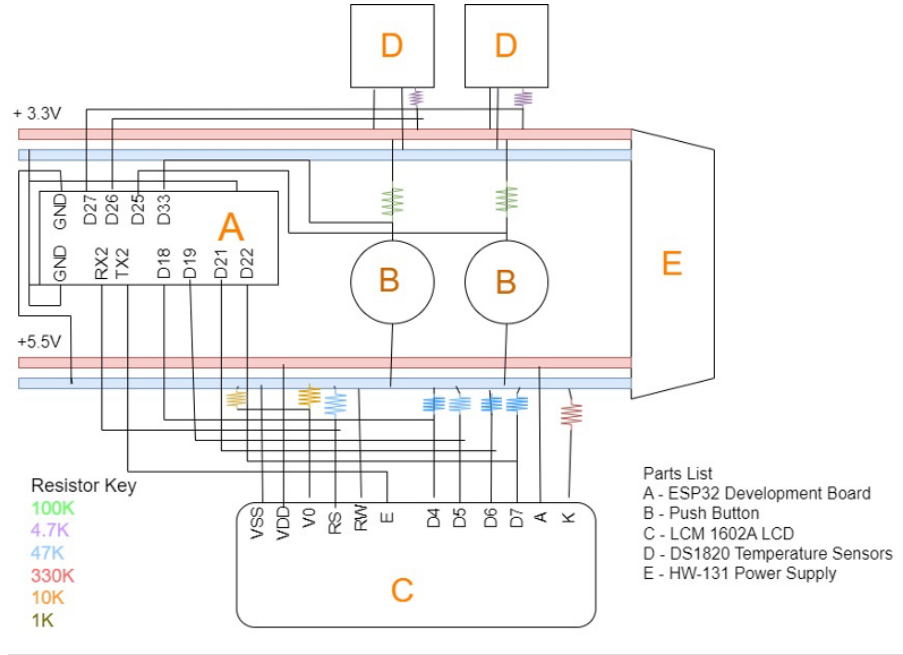


Figure 1.2: System Hardware Wiring Schematic

1.3 Software Design: ESP32 Firmware

This section outlines the basic software design for the firmware running on the ESP32 that interacts with the temperature sensors, push buttons, and LCD.

1.3.1 ESP32

We used ESP32 version 2.0.17 by Espressif Systems as the base firmware [14]. As we explain later in this report, using an older version was a deliberate design choice.

1.3.2 Main Control Loop

Per the lab requirements, the ESP32 must be able to control a system consisting of two temperature sensors, two push buttons, and an LCD. The ESP32 firmware runs a continuous loop that checks if each sensor has been turned "ON" or "OFF" through the push button toggles or client and then displays the temperature or sensor status to the LCD. If a sensor has been toggled "OFF", the current temperature is set to -10000.0, and "OFF" is displayed on the LCD.

When the temperature is being read from a sensor, the connection status is also verified. If the sensor is disconnected, the temperature is set to -11000.0, and "Error" is displayed on the LCD. This loop repeats every 50ms and updates global variables that store the current temperature and status for each sensor on every iteration. The pseudocode for this process can be found in [1].

1.3.3 Interrupts

We use falling-edge interrupts to capture user I/O via the push buttons. There is an interrupt for each button, and when the interrupt is triggered, the status of whichever button triggered it is negated. This allows us to simulate the "ON" or "OFF" state of each sensor. The pseudocode for this process can be found in [2].

1.4 Software Design: ESP32 Web Server

This section outlines the basic software design for the web server code that runs during the initialization of the ESP32 firmware.

1.4.1 Setup

Upon initialization, the ESP32 firmware attempts to set up a web server. If the web server is successfully initiated, the current dynamic IP address is output to serial and sets up the following endpoints:

- `/temperature1`: Requires the parameter `?unit=<unit>` where `<unit>` specifies the temperature unit (C or F).
- `/temperature2`: Requires the parameter `?unit=<unit>` where `<unit>` specifies the temperature unit (C or F).
- `/toggle1`: Requires the parameter `?toggle=<toggle>` where `<toggle>` specifies the toggle state (ON or OFF).
- `/toggle2`: Requires the parameter `?toggle=<toggle>` where `<toggle>` specifies the toggle state (ON or OFF).

If the web server fails to initialize, the system operates in "offline" mode, retaining all previously discussed functionality from Section 1.3.

1.4.2 HTTP Interactions

As defined in Section 1.4.1, the web server sets up 4 endpoints. The **temperature** endpoints are used to retrieve the current temperature of a given sensor and update the current unit (See [5.1] and [5.2]). The **toggle** endpoints are used to toggle a given sensor "ON" or "OFF". Upon receiving a message at one of these endpoints, the server uses the parameter (either **unit** or **toggle**, depending on the endpoint) to return or update a global variable in the ESP32 firmware. This allows for remote control of the button status and unit by the client and also allows for real-time retrieval of the current temperature off either of the two sensors. The pseudocode for this process can be referenced in [3] and [5].

1.5 Software Design: GUI

We used the Flask Python library [7] to develop a front-end application within the ‘GUI.py’ file which displays the interface on localhost. Within this file, we utilize multiple routes to communicate with the server, from fetching and sending information. Using HTTP GET requests, the application serves as a client to the server located on the ESP. We pass parameters with the GET requests; for example, we specify the scale (C or F) in which the temperature should be reported from the sensors. The pseudocode can be found in [4].

The main application route, ‘/’ renders the template HTML file, ‘gui.html’ which also contains our CSS styling and JavaScript functionality. Within the template file, we use a JavaScript script and its jQuery library to handle events such as temperature plotting and handling user input without reloading the page.

1.5.1 Routes Interacting With Server

Many routes within the application require contact with the server, they are as follows:

- `/temperature`: Used for fetching the temperature from the server.
- `/toggle1`: Used for toggling sensor 1.
- `/toggle2`: Used for toggling sensor 2.

Every second, the `fetchTemperatureData()` JavaScript function polls the temperature from both temperature sensors using the GET `/temperature` application route. These routes perform two functions, requesting the temperature from the server and returning the result to the GUI in addition to handling the maximum and minimum temperature alerts. Once the temperature is returned, it is formatted, updated for both `temperatureStatus` elements, and sent to be placed on the chart. Our implementation of the chart uses the open-source `Chart.js` JavaScript library. We encountered issues when attempting to create the scrolling effect for the graph which we eventually implemented by making an additional chart for scrolling and adjusting HTML and CSS elements once temperature data extends further than the last 300 seconds. The GUI also allows clients to turn off the sensors from the front end. To implement this feature, we need to send the server a message which is done using a toggle button for each sensor. The message sent to the server is a GET request to the `/toggle1` and `/toggle2` routes that take a parameter “toggle” which can be either “OFF” or “ON”. Switching from Fahrenheit to Celsius can be done using a different toggle button located near the sensor toggle buttons. The `/switch_scale` route switches a global unit variable between ‘C’ and ‘F’. When this unit variable changes, the unit variable in the temperature fetch requests is also changed allowing us to retrieve the data from the temperature sensor in both scales.

1.5.2 Routes Without Server Interaction

Other routes within the application do not directly interact with the server. These routes include:

- `/max_min`: Used to set the maximum and minimum temperatures.
- `/user`: Used for setting user information.
- `/switch_scale`: Used to switch temperature scale.

The `/user` route is used when a client/user provides the necessary contact information for sending emails and SMS messages. This route sets the email, phone number, and cell carrier instance variables for the Client class object. When the temperature passes the minimum or maximum temperature an email and a text message (for AT&T, Verizon, T-Mobile, Sprint, and US Cellular customers) will be sent to their device. Python's easy-to-use SMTP library [8] allows the application to send emails and SMS messages through a Gmail SMTP server. Recently, changes to Gmail account security features require applications that require Gmail account sign-ins to either use a secondary 2-factor authentication method or create an app password which is required when signing in with that application. We do not anticipate security vulnerabilities to be taken advantage of with our application allowing us to use the less secure app password. Maximum and minimum temperatures for sending client messages can be set through the `/max_min` route in the GUI, which simply updates the Client object's instance variables for temperature limits. When this route is called, the Client object's number of messages variable is also reset allowing for another message to be sent out to the receiver.

1.6 Software Design: Schematics

Below is the high-level software design for the entire system. One key feature is that the web server, while being run on the ESP32, acts as a bridge between the ESP32 firmware and the client software. Additionally, both the embedded system and the client can update global variables through web server interaction. This feature allows for sensors to be turned on or off through push button I/O and also remotely through the GUI. Furthermore, this allows the GUI to update the current temperature's unit of measure on the embedded system.

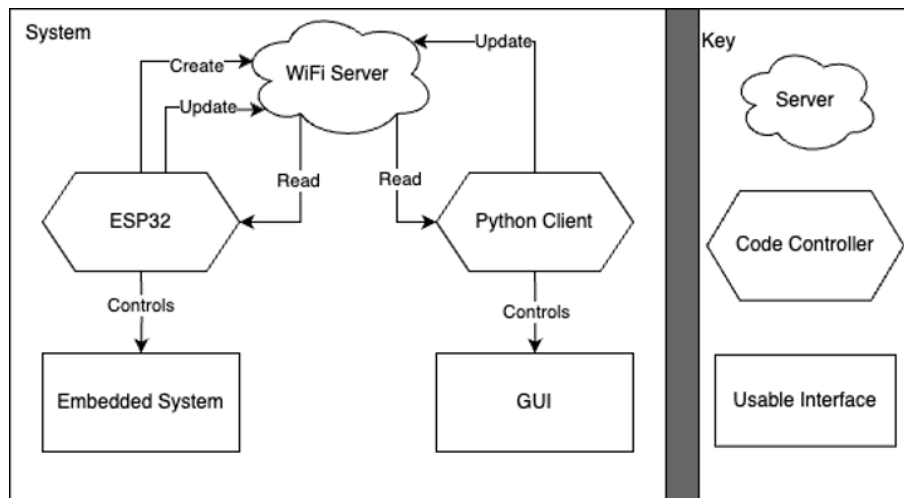


Figure 1.3: High-level Software Design Schematic

Below is a more detailed version of the software design for the system, including program control loops for both the embedded system and the client. The only connection between the ESP32 and the client is through HTTP requests. The ESP32 continuously updates temperature data, regardless of whether the sensors are turned on. This paradigm allows for a wireless connection between the client and server with real-time data transfer. Furthermore, this framework allows all of the control global variables (sensor toggle statuses, measurement units) to be controlled by the server, user I/O, or both at the same time. Thus, the normal operation should continue uninterrupted even without a Wi-Fi connection.

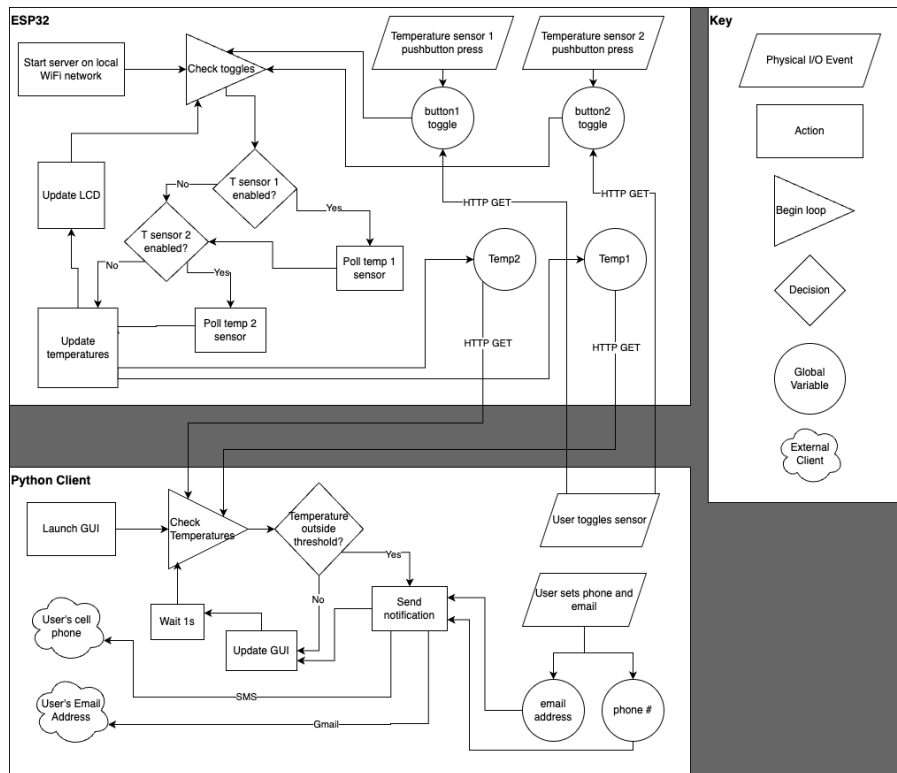


Figure 1.4: Low-level Software Design Schematic

Chapter 2

Design Process and Experimentation

This chapter provides details about the design process of this lab, including the challenges we faced and the solutions we developed to overcome them.

2.1 Hardware Selection: "Third Box" Circuitry

This section describes details about the development of the "Third Box" circuit design.

2.1.1 Hardware Constraints

The two waterproof DS18B20 digital thermometers and one major component of the "third box," an ESP 32 development board, were pre-determined and essentially functioned as hardware constraints, compelling an evaluation of their characteristics to occur first.

A cursory reading of the DS18B20 datasheet [12] shows a digital sensor that communicates along a one-wire bus and is powered at a relatively large range of 3 to 5.5 volts which can either be sourced externally or parasitically along its data output line. The decision to power the temperature sensors from an external supply came easily, as it simplified the final system design in eliminating the need for appropriate switching of a pull-up MOSFET during times of sensor temperature conversion.

An initial reading of the ESP 32 development board's ESP 32 WROOM 32 module and chip datasheets [13][14], meanwhile, reveals significant capability and versatility, including processing speeds up to 240 MHz, Wi-Fi and Bluetooth functionalities, transmitting and receiving antennas, and low power consumption modes, all at an operating voltage of 3.3 volts.

Another major "third box" constituent, as outlined in the design requirements, is the display. The length of the text required by the display along with a preference for a side-by-side comparison of the two DS18B20 thermometer inputs deemed the use of seven-segment LEDs as a display too complex and burdensome. Another display option, the HD44780 driver-compatible [17] LCM 1602A LCD [16], a 2x16 character display with contrast, backlight, and a four-control line option driven by 5 volts, was a more natural design choice both in its capability and in its

ready availability.

2.1.2 Voltage Differences

With a general understanding of ESP32, DS18B20, and LCM 1602A operation, it was evident that differences in their operating voltages needed to be resolved. To confirm the purported voltage range of the DS18B20 sensors and to verify their operation at both 3.3 and 5 volts, a simple program that retrieved and displayed their temperatures was uploaded to both the ESP 32 and an Arduino Uno R3. Both the microcontrollers, the Arduino Uno receiving data from a 5-volt connected DS18B20 and the ESP 32 receiving data from a 3.3-volt connected DS18B20, recorded similar temperatures with similar accuracies relative to an HVAC thermostat display. The DS18B20 temperature sensors thus proved to perform similarly at 3.3 and 5 volts and could be set aside while the task of accommodating the differing voltage requirements of the LCM 1602A and the ESP 32 was addressed.

There were several different methods by which to resolve the voltage requirements, including

1. Sourcing a 3.3 volt LCD
2. Using the ESP as only a Wi-Fi module secondary to a 5-volt primary processor which could interface with the LCM 1602A
3. Employing hardware to enable the use of I2C communication between the ESP 32 and the LCM 1602A instead of relying on four data lines
4. Directly interfacing the LCM 1602A and ESP 32

Options 1 and 3 required procurement of additional hardware and so took a back seat to options 2 and 4. Option 2, powering the ESP 32 as a Wi-Fi module through the low dropout (LDO) linear regulator of a different development board seemed unnecessarily complicated, expensive, and wasteful so finding a practical method by which to connect the LCM 1602A and ESP 32 directly became the focus of the design.

To test and verify that the LCM 1602A would not operate correctly at 3.3 volts, both its power supply, Vdd, and contrast, Vo, pins were supplied that voltage. The backlight remained adequately illuminated, but the character display was no longer visible as it had been when the LCM 1602A pins were supplied with 5 volts. The test confirmed that the LCM needed to be provided 5 volts.

Before testing the operation of a 5-volt powered and 3.3-volt ESP 32 output-controlled display, open circuit voltage measurements were taken on all the LCM 1602A control and data lines while its Vdd and Vo pins were supplied 5 volts. All the control lines, save one, measured approximately 5 volts, evidence that the LCM 1602A inner circuitry was pulling its pins high. To dissipate any current that might be driven from these pulled high LCM 1602A pins to their corresponding ESP 32 output pins when the ESP 32 issued digital high commands, the pulled high LCM 1602A pins were connected to the ground through resistors. The appropriate values for these resistors were determined by trial and error, open circuit voltage measurements of

their connected pulled high LCM 1602A pins were measured until they were approximately 3.2 volts. This design addition would serve to limit the times when the ESP 32 sinks current to when its output control lines are digitally low.

After uploading simple code to the ESP 32 to control the LCM 1602A and powering both with their respective voltage requirements, the LCM display seemed to function as instructed. Should this design eventually prove insufficient, viable alternative solutions included the addition of a level shifter or negative charge pump circuit to provide a 5-volt differential with a high of 3.3 volts. With that said, the system design now turned into a suitable power supply for both 3.3 and 5 volts.

2.1.3 Power Requirements

An on-hand power supply module that uses LDO linear regulators [15] to provide both 3.3 and 5 volts is the HW-131 breadboard power supply module included in the RexQualis Arduino starter kit. Although this module perhaps isn't intended to be used in more permanent installations, it offers a convenient way to incorporate both a wall-wort supply connection for development and an ultimate battery connection for the final prototype in addition to producing 3.3 and 5 volts. Alternative power supply output circuits compromised of individual LDOs and capacitors would allow for a more compact and solderable design but in a less convenient form, while those including a buck-boost converter would offer precision and adjustability but at the expense of design simplicity.

As reassurance that the HW-131 would be able to accommodate the power requirements of the "third box," a back-of-the-envelope calculation was made. According to its datasheet, the A1117 LDO can dissipate a maximum of 500 mW. For the power supply's primary LDO, which has a 5-volt output, this maximum power dissipation value allows for 125 mA from a 9-volt battery. According to the ESP 32 WROOM 32 datasheet, the Wi-Fi module's consumption varies around 100 mA, which added to the approximately 30 or so mA of the LCM 1602A backlight would hover at the 125 mA HW-131 rating. Considering that the power supply is temperature protected and has a maximum current rating of 1 amp and that the ESP Wi-Fi module would likely only be intermittently sourcing 100 mA, the decision was made to include the HW-131 power supply in the system design.

As has been a consistent theme in the design of the "third box," the choice of a 9-volt battery for the final prototype power supply was made primarily out of concern for convenience and available materials. Series-tied AA batteries with a total voltage closer to the 5-volt output of the power supply module would be more efficient and reduce the LDO's dissipation load but would require batteries and a mounting method different from the 9-volt setup.

The requirement satisfaction for a "third box" on/off switch naturally settled on the toggling of the 9-volt battery's positive leg to minimize current draw during times of intended system inoperability. It also, however, elicited questions as to what turn-on starting mechanism, if any, needed to be included in the system design to protect components. The ultimate decision to exclude any soft start or similar circuitry to limit in-rush current and slowly ramp up supply

voltage was made based primarily on two pieces of anecdotal evidence. Firstly, the HW-131 power module includes an on/off switch, and yet its datasheet doesn't explicitly mention soft start capabilities (although perhaps this is implicit in its circuitry). Secondly, when asked about the need for a special starting mechanism, Professor Krueger, knowledgeable in embedded system design, mentioned that the only circuits in which he's seen such a design included very large capacitive loads.

With power supply components settled upon and implemented, testing of the "third box" revealed erratic temperature readings while the DS18B20 sensors and the ESP 32 were both supplied 3.3 volts. The subsequent shifting of power input to the ESP 32 to its 5-volt LDO pin seemed to resolve the issue.

2.2 Hardware Selection: "Third Box" Construction

This section provides details about the construction of the device, specifically the "Third Box."

Initial design of the physical construction of the "third box" centered around wood as a medium. Wood box construction allows for custom dimensions, rugged design, and accessible elements, including the LCM 1602A display, to be recessed below the outermost surfaces of the "third box" where they would be protected from impact. Drop tests conducted on a preliminary wood box, however, revealed greater energy transference to elements inside of it compared to those inside plastic and cardboard boxes subjected to similar tests. Eventually, it was decided that a small plastic container would be used as the physical "third box" as it has several advantages, including that it is flexible upon impact, it is easily drilled through, it bonds well with hot glue and it is transparent, allowing the LCM 1602A display to be seen while mounted inside the box.

Physical circuit construction is based on back-to-back breadboards primarily to ensure that the HW-131 and ESP 32 remain easily transferrable and because the width of the ESP 32 demands more than that of one breadboard. As revealed during drop tests, the double-sided insulating tape on the breadboard undersides is surprisingly strong and adheres well to a variety of substrates. A thin piece of plywood was chosen as a base for the breadboards and attached to the plastic container with metal hardware, a foam cushion, and thick rubber washers.

Concern remains, however, about the ability of the connections on the breadboard to undergo drops without becoming loose. Well-placed zip ties, dabs of hot glue, and wire tie-downs serve to ameliorate some of this concern and have proved effective during drop tests performed on the fully constructed prototype design. Assuredly a better option for connections in future developments of prototype design is solder, but one that would require desoldering for component reuse.

The DS18B20 sensors are connected to the "third box" with plastic, three-pin, twist-lock, bulkhead-type connectors [18] [19] that are of a rugged design, allow for easy removal of the sensors, and achieve a very positive mounting to the side of the "third box" with a nut and threads

2.3 Software Selection: ESP32 Firmware

When developing the ESP32 firmware code, we installed and used the following packages from the Arduino IDE library manager:

- LiquidCrystal [1] is used for interfacing with the LCD
- DallasTemperature [2] is used for interfacing with the temperature sensors
- OneWire [3] is used for interfacing with the temperature sensors

We chose the aforementioned packages to interface with our temperature sensors and LCD because they streamlined the process of reading temperatures and writing data to the LCD. After researching these packages, we found that they were stable for the ESP32 board version we were using. Additionally, they made interfacing with each hardware component straightforward—for example, initializing a LiquidCrystal object only requires specifying the LCD pins. Using these packages removed the complexity of having to reinvent the wheel of using extremely common hardware components. This allowed us to devote the majority of development time to setting up the logic for the web server, which ended up being much trickier.

2.4 Software Selection: ESP32 Web Server

When developing the ESP32 web server code, we installed and used the following packages from the Arduino IDE library manager:

- WiFi [4] is used for wireless connectivity for the ESP32
- AsyncTCP [5] is used for web server initialization

These libraries allowed us to set up a web server on the same local Wi-Fi network as the client. This allowed us to use HTTP as the primary client-server interaction protocol. We decided on HTTP because it best fits our needs of sending/receiving small packets of data (floats or strings) and has minimal complexity.

2.4.1 Enterprise Wi-Fi Connection (eduroam)

The first issue we encountered was setting up the web server on eduroam, which is an enterprise Wi-Fi network with higher security than a personal Wi-Fi network. After extensive debugging with the WiFi and AsyncTCP packages, we discovered that connecting to enterprise Wi-Fi using the latest ESP32 board version (3.0.4) was convoluted and difficult. After downgrading the ESP32 board base firmware to 2.0.17, we were able to connect to enterprise Wi-Fi much easier using an older version of the Wi-Fi package. Furthermore, this downgrade did not effect on the rest of the software package dependencies, so we opted to use ESP32 version 2.0.17 as our baseboard.

2.4.2 Client-Server Communication

After successfully setting up the web server on eduroam, we needed to figure out the best method of setting up the actual communication between the client and the server. The main issue that we needed to solve was how to determine the dynamic IP of the web server from within the client software. We tested three different methods, outlined below.

Method 1: Hard-Coded Dynamic IP

The first method we tried was hard-coding the web server’s dynamic IP into the client code, directing client requests straight to the server’s IP. How it works:

1. Connect the device to the PC via USB
2. Reset ESP32
3. After the Wi-Fi connection, ESP32 outputs the current dynamic IP over serial
4. Hard-code the observed IP into the client software
5. Disconnect USB from PC

While it is a pain to set up the first time, the dynamic IP should not change until the device disconnects and reconnects to Wi-Fi again. This ensures that after the initial setup, no further configuration is needed unless the device reconnects to Wi-Fi. This method yields extremely fast server response times since client HTTP requests bypass DNS routing. The response times for this method were observed to be within 150ms per request.

Method 2: DNS Domain Name

The second method we tried was to skip the necessity of running with a serial connection on initial start-up by setting up a DNS domain name using the ArduinoMDNS package provided by George Kaindl. This allowed us to set the server domain name “esp32.local” on the Wi-Fi network, which the client could then send HTTP requests to. While this was much more efficient in terms of setup, response times were much slower than the previous method, and there was no guarantee that a response time would be less than 1s. This violated one of the requirements of the lab, and thus we could not proceed with this method.

Method 3: UDP Broadcasting

The last method we tried was to broadcast the dynamic IP of the web server over UDP to the client using the WiFiUDP package [6]. How it worked:

1. Broadcast current dynamic IP from ESP using UDP on a predefined multicast port and IP (we used 12345 and 238, respectively)
2. On the client, listen to that location for the broadcasted IP
3. Once the IP has been read, proceed with connection

This would allow the client to listen on a socket in the background for the UDP message, and connect once the message is read. This worked well on local networks, such as a home network, but the client was not able to find the UDP message on enterprise networks such as eduroam. Therefore, this solution didn't add anything because we still would need to hard-code the IP on eduroam regardless, so we did not proceed with it.

2.4.3 Final Design

In the final design, we selected Method 1. While this approach isn't the most convenient, it's manageable. If this thermometer were developed into a real product, users would likely need to complete an initial setup process, such as configuring the Wi-Fi credentials. In this case, connecting to their PC during the first startup wouldn't pose too much inconvenience.

2.5 Software Selection: GUI

The graphical user interface (GUI) leverages the Flask pip package to create a front-end interface for clients to interact with the server hosted on the ESP32. Messages going between the server and client are bare bones containing little information, at times just a single parameter. This led us to use simple HTTP GET requests for server/client communication and application routing. Many options were available for implementing this feature however, factors such as complexity, ease, and interest were considered throughout the process.

2.5.1 Initial Thinking

Initially, the question for GUI development was what language to develop with, either Python or C++. Python was chosen over C++ due to its ease of use and its support with a variety of packages. We thought to create a GUI using the tkinter package [11] however we encountered problems with limitations regarding the html elements allowed within the tkinterhtml extension. These problems prompted us to use a Flask application that fetches data from the ESP-hosted server and displays the temperature updates by adjusting the HTML and JavaScript elements within. Flask was chosen over Django due to its user-friendliness and its ease of use with smaller and less complex applications. Templating within Flask was extremely beneficial when first learning how to use the package and the documentation was easy to understand.

2.5.2 Implementation

The implementation of the graph in JavaScript uses the following packages:

- Chart.js [9] is used to create the graph seen on the interface
- jQuery.js [10] is used for handling routes without reloading the page

Within JavaScript, we utilize the Chart.js library for plotting temperatures because of its robustness and plentiful documentation. JavaScript also allowed for an easy implementation of handlers for HTML elements such as the form submissions and the toggle buttons. Users interacting with the interface will receive alerts containing information about their requests.

Implementing these updates without jQuery and using exclusively HTML/CSS required us to reload the entire page. This was not only useful for debugging but also gave the user a responsive experience enhancing usability. To create a scrollable chart, Chart.js does not provide a feature directly in their documentation for this purpose which required us to implement the feature through adjusting HTML and CSS. Other potential solutions for this problem we considered were looking for other options in JavaScript such as Platyly.js or using Python's Matplotlib package for creating the graph but embedding this continuously updating graph proved challenging. Another option we considered but ruled out was using a C++ interface because of the initial thought that the complexity was not worth the loss of time we could spend on other areas of the project.

2.5.3 User Messaging

To send notifications to users, we used the SMTP Python library [8] for both SMS and Email. Our implementation used Gmail as the SMTP host. We considered using the Twilio library for sending SMS messages but due to the cost of sending messages this was ruled out. Twilio also is much less user-friendly compared to our use of the SMTP library. The SMTP library is not only free but can be implemented with ease, which is the reason for our choice. With the ease of implementation, there was one drawback, SMS messages can only be sent to numbers with the AT&T, Verizon, T-Mobile, Sprint, and US Cellular carriers which allow for email to SMS messaging.

Chapter 3

Test Report

This chapter presents the test reports for the hardware and software components of the device, verifying that our device fulfills the requirements of the lab.

3.1 Test Strategy

We employed a hybrid testing strategy, using a mix of manual and automated tests. The hardware and holistic system were primarily tested manually, with results being observed in real time. The software tests had a mix of manual and automated regression testing. For functions that are intended to stay unchanged throughout software updates, unit tests were written and added to a series of bash scripts that were run by GitHub actions on each new commit to the main branch.

3.2 Hardware Tests

Table 3.1: Hardware Test Report

Test	Description	Method	Result
Verify temperature sensor is water-proof	No damage when placed in ice water	Manual	Pass
Withstand fall damage	Withstands fall from bench, cables and connectors may unplug, but do not break	Manual	Pass
Operation while upside down	Maintains normal operation while upside down, cables stay plugged in, displays correctly	Manual	Pass
Connectors easily connected	Secure connection, able to disconnect quickly and efficiently	Manual	Pass
No intervention if sensor disconnected	Maintains normal operation when unplugged	Manual	Pass
Switch is off, Box operation	Box cannot display or connect to the internet, nothing displayed on LCD, no data displayed on computer	Manual	Pass
LCD unplugged error message	LCD should display error message for Sensor1/Sensor2 if unplugged	Manual	Pass
Switch on, sensors at startup	Displays off messages for sensors, "Sensor1 off/ Sensor2 off" on respective lines on startup	Manual	Pass
Correct temperature display	Displays correct temperature, noticeable delay greater than 20 ms, buttons pressed	Manual	Pass
LCD visibility	Intensity of the backlight is set to a level visible in indoor lighting	Manual	Pass
Data and off message accuracy	Correct message displayed during all button states, whether both on, just one, or none	Manual	Pass

3.3 Software Tests

Table 3.2: Software Test Report

Test	Description	Method	Result
Computer displays temperature	User can control units (C or F), updated every second, both temperatures displayed	Manual	Pass
Unplugged sensor message on computer	Computer should display “sensor unplugged” instead of displaying temperature	Manual	Pass
No data available message on computer	Computer displays “no data available” message when relevant	Manual	Pass
Disable temperature display from computer	User can stop displaying the temperature from computer interface	Manual	Pass
Graph display on computer	Graph of the past 300 seconds displayed on computer, with user control for units (C or F)	Manual	Pass
Start up timing on computer	Data should display within 10 seconds of box being turned on, when computer is connected	Manual	Pass
Temperature alert message	Email sent if the temperature exceeds maximum or falls below minimum threshold	Manual	Pass
Computer edit ability	Phone number/email, messages, and temperature thresholds can be edited from the computer	Manual	Pass
Temperature value accuracy	Temperature should be within $22 \pm 4^{\circ}\text{C}$ for room temperature, and $0 \pm 4^{\circ}\text{C}$ for ice water mixture	Manual	Pass
Temperature calculation accuracy	Temperatures are able to be converted between C and F	Automated	Pass

Table 3.3: Software Test Report (continued)

Test	Description	Method	Result
Endpoint response time	Message sent to each endpoint 10 times and verify the average response time is less than 1s	Automated	Pass
Verify HTTP remote toggle	requests are made to the /toggle endpoints and verify whether the sensors are correctly toggled	Automated	Pass
Verify HTTP temperature response	requests are made to the /toggle endpoints and /temperature endpoints to verify whether temperatures are being correctly returned for all button states	Automated	Pass
Invalid endpoint test	Verify that making a request to an invalid endpoint receives an error code and does not interrupt device operation	Automated	Pass

Chapter 4

Project Retrospective

This chapter reflects on the outcomes of the project, discussing project management strategies and possible improvements for future projects.

4.1 Project Management

As a team, we employed a hybrid development model for the lab project management. We adopted an agile approach with rapid iterations for software development, while the hardware development followed a more waterfall-oriented approach. We chose this development model because it allowed us to optimize the time spent sharing the device throughout development. Our development model was as follows:

1. Develop hardware to requirement standards (waterfall)
2. Once in a stable state, pass off for software development
3. Develop software to requirement standards (agile)
4. Convene to reflect on progress, devise solutions to problems, and test the system
5. Repeat

This model gave the hardware-focused members of the team ample time and ability to develop the hardware up to the standard of requirements, while also giving the software-focused team members the freedom to rapidly iterate on stable hardware. Overall, this model played to the strengths of our team and allowed us to develop fast and efficiently.

4.2 Gantt Chart

As a team, we kept a Gantt chart up-to-date throughout the entire design process to keep track of our objectives list and timelines for completion. The chart is displayed below.

Holland, Bri, Joseph, Cavan

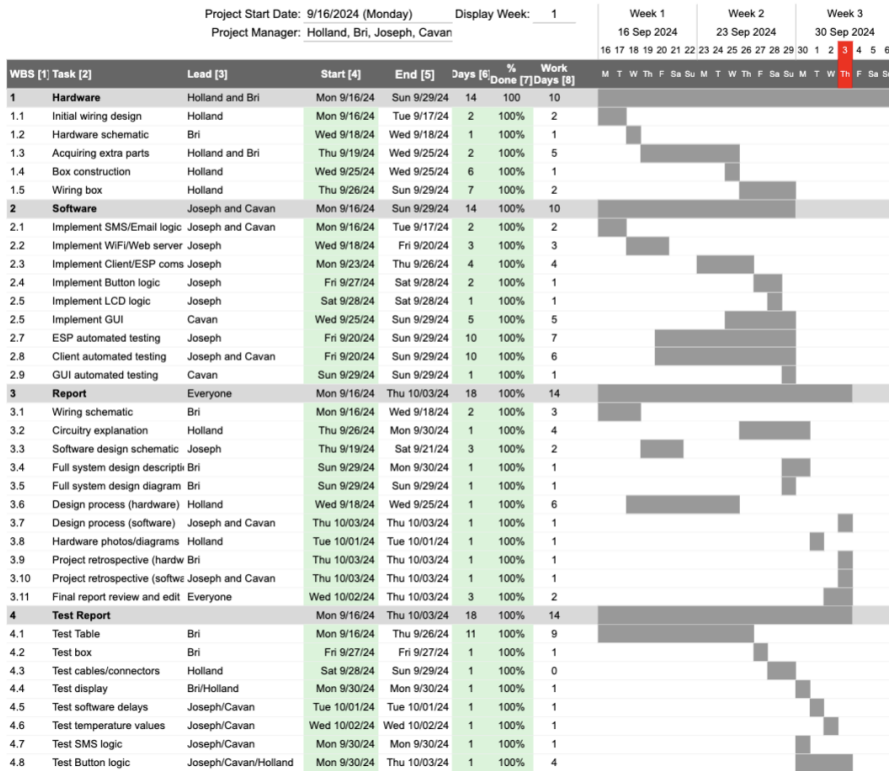


Figure 4.1: Lab 1 Gantt Chart

4.3 Outcome Summary

All in all, the final lab check-off tests of our smart thermometer design were successful. There were a few issues, however, that would need to be addressed in future prototypes.

4.3.1 Hardware

During the final lab check-off the LCD's light intensity was pulsating, indicative of insufficient power. Although providing sufficient voltage levels during testing, perhaps the 9V battery and HW-131 combination were unable to meet the periodic power requirements of WiFi transmission.

Additionally, one DS18B20 sensor responded slowly to temperature increases, possibly due to power quality issues or a faulty sensor. This sensor had exhibited problems throughout prototype testing. Comparing the system's performance with a new sensor would be beneficial.

4.3.2 Software

We were informed during the lab checkoff that our understanding of the "scrollable" graph was incorrect, and that scrollable meant the graph window was moving to keep only the last 300s in frame. We understood that requirement to mean that 300s were shown in the frame and that the user could scroll back in time to see old data. So, we should have reached out for clarification

on that requirement. Furthermore, this would've decreased the software complexity of the GUI significantly.

Moreover, the Gmail SMTP implementation for SMS/Email notifications proved unreliable, with unexpected server connection drops. For a production system, we would consider using the Twilio API or another enterprise solution to ensure consistent uptime and timely notifications.

4.4 Possible Improvements

This section outlines possible improvements for the hardware and software components of the system.

4.4.1 Hardware

While our hardware is sufficient, multiple things could be improved. If we had used higher-quality buttons, we would have less problems with bouncing. As for the box design, we could have been more creative and made something from scratch to make it look cleaner and therefore more appealing to the user. With more time, we would have been able to test multiple materials and how efficiently they absorb the fall and select the best option. Additionally, substituting a soldered prototype board and a permanent power supply for the breadboards and modular HW-131 supply would result in a more compact and rugged "third box." With pre-drilled holes, a soldered prototype board could be elevated above and connected to the surface of a variety of substrates with substantial rubber cushioning and metal hardware.

4.4.2 Server

A key improvement would be streamlining client-server communication, particularly in handling the web server's dynamic IP (See Section [2.4.2]). As it stands, our current method of hard-coding the IP is cumbersome. A potential solution, which we didn't have time to explore, would be to use AWS Secrets Manager to store and manage the server's IP and Wi-Fi passwords. This approach would eliminate the need for hard-coding sensitive information in the system software.

4.4.3 GUI

With the knowledge we learned throughout developing this lab, we saw many potential areas of improvement regarding the user interface. While the user interface currently is an acceptable implementation, we believe there to be improvements in both speed and smoothness. The GUI could potentially be moved to C++ for improved memory management and the inherent speed advantages of C++. This, while being more complex, would have the trade-off of being a potentially better implementation.

4.5 Member Contributions

This section outlines the individual contributions of each team member.

4.5.1 Cavan

Cavan took the lead in the development of the GUI and testing as well as collaborating with Joseph on designing the SMS/Email service. Additionally, he contributed to multiple design choices across the software in this project.

4.5.2 Joseph

Joseph led the development of the ESP32 firmware, web server code, and the automated testing suite for the device software. Furthermore, he contributed to the software system design schematics and various design choices for the device software, specifically with client-server interaction and SMS/Email services.

4.5.3 Bri

Bri was responsible for providing a wiring schematic and a system diagram. She completed the design documentation section where she gave a summary of the hardware requirements and an overview of the system as a whole using the schematics.

4.5.4 Holland

Holland was responsible for the design, construction, and testing of the "third box." He also completed the design process and experimentation section of the project report and contributed hardware photos.

Chapter 5

Appendix

This Appendix outlines the figures, pseudocode, and references used in the project.

5.1 Figures

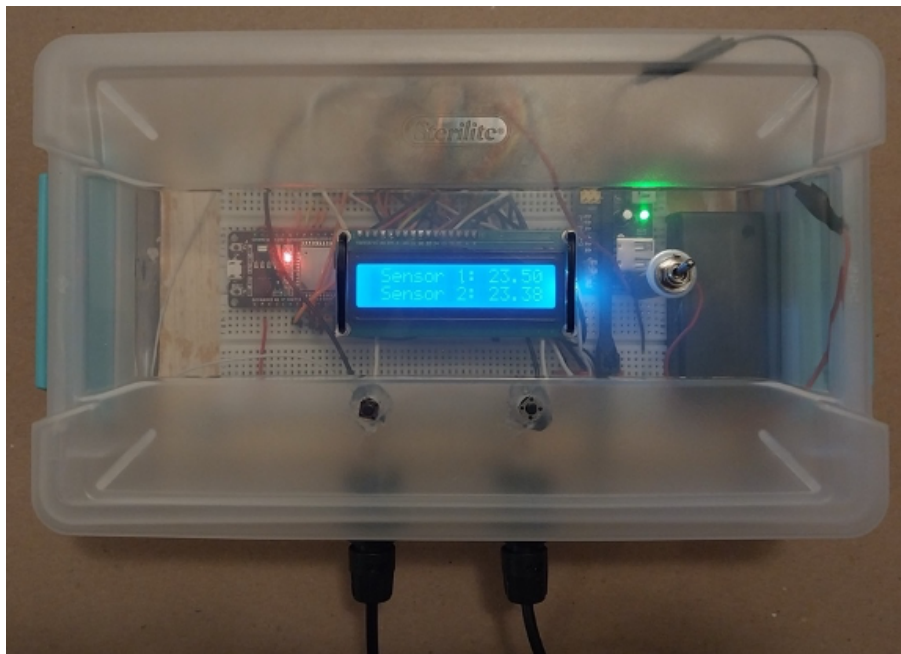


Figure 5.1: Device Box

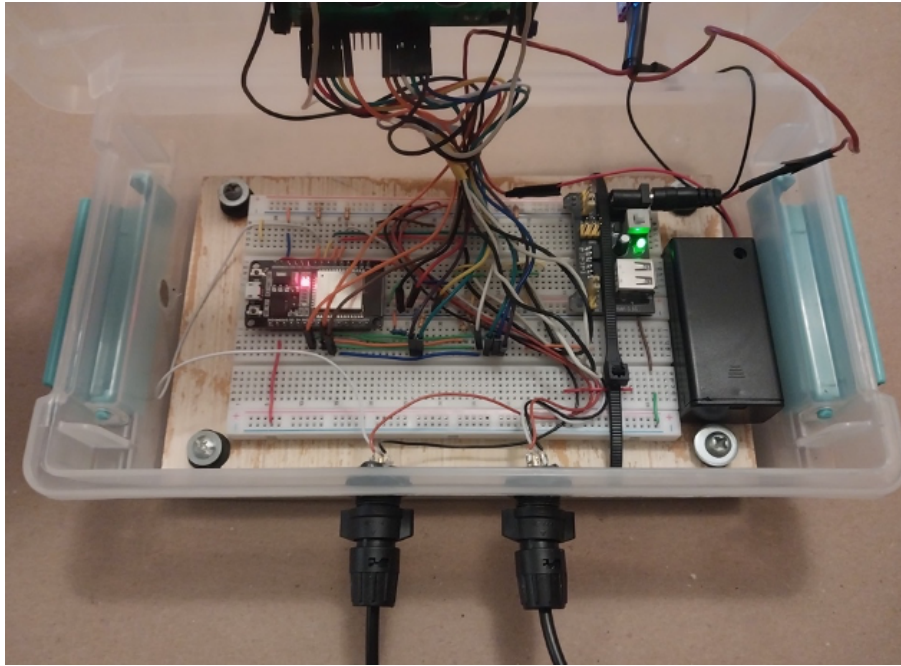


Figure 5.2: Device Circuitry



Figure 5.3: Temperature Sensor Connector

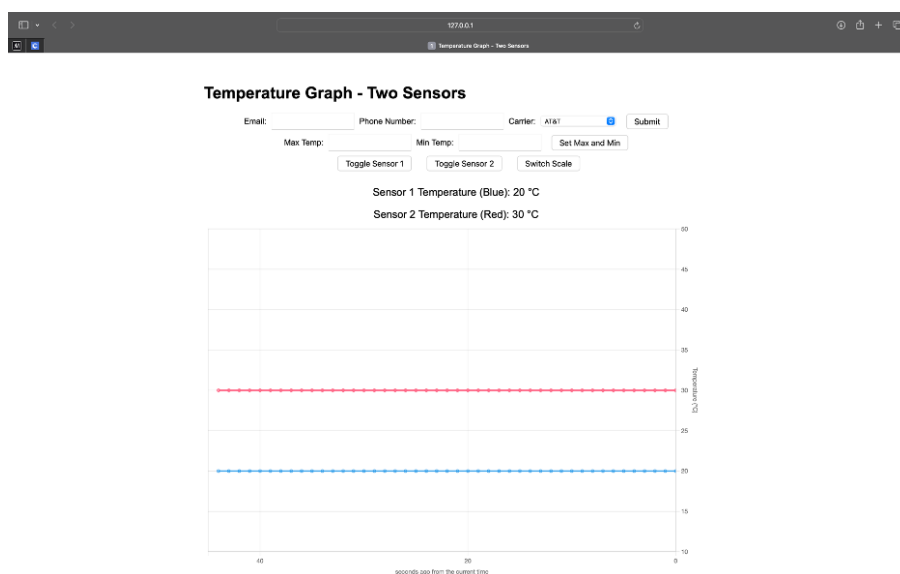


Figure 5.4: The Client Graphical User Interface (GUI)

5.2 Code

Full codebase: <https://github.com/ECESeniorDesign24-25/SmartThermometer>

Download the codebase [here](#).

Below is the pseudocode for important processes in the system software.

```
1 curl http://<ESP32_IP>/temperature1?unit=C
```

Listing 5.1: ESP32 HTTP requests for returning temperature in Celsius using curl

```
1 curl http://<ESP32_IP>/toggle1?toggle=OFF
```

Listing 5.2: ESP32 HTTP request for toggling a sensor off using curl

Algorithm 1: Main Control Loop

Global *temp1, temp2, sensor1, sensor2, unit, button1Pressed, sensor1Enabled, button2Pressed, sensor2Enabled*

while *True* **do**

if *button1Pressed = True* **then**
 | *sensor1Enabled* $\leftarrow$ $\neg$ *sensor1Enabled*

end

if *button2Pressed = True* **then**
 | *sensor2Enabled* $\leftarrow$ $\neg$ *sensor2Enabled*

end

temp1 = *getTemperature(sensor1, unit, sensor1Enabled)*;

temp2 = *getTemperature(sensor2, unit, sensor2Enabled)*;

 Display *temp1* to LCD;

 Display *temp2* to LCD;

 Wait 50ms;

end

Algorithm 2: Push Button Interrupt (Falling Edge)

Global *debounceDelay, lastButtonPress*

if *currentTime - lastButtonPress > debounceDelay* **then**

 | *buttonPressed* $\leftarrow$ **True**;

 | *lastButtonPress* $\leftarrow$ *currentTime*;

end

Algorithm 3: /temperature Endpoint

Input: httpRequest**Global** *currentTemperature, sensorEnabled, sensorOffTemp, unit**newUnit* $\leftarrow$ **unit** parameter from httpRequest;**if** *newUnit* = 'C' **or** *newUnit* = 'F' **then** **if** *sensorEnabled* = **then** Convert *currentTemperature* if necessary; **end** **else** *currentTemperature* $\leftarrow$ *sensorOffTemp*; **end** *unit* $\leftarrow$ *newUnit*; Send *currentTemperature* over HTTP;**end**

Algorithm 4: GUI Overview

Global *sensor1Status, sensor2Status, unit, OffTemp, UnpluggedTemp*

GUI initialized on localhost when 'GUI.py' is ran

// The temperature is fetched from the server using the /temperature application route
and added to graph.**if** *temperature* = *OffTemp* **or** *temperature* = *UnpluggedTemp* **then** *temperature* is set to None;**end**

// jQuery handlers monitor routes that require user input:

1. /user: sets client class variables to the entered contact information.
 2. /max_min: sets the client class maximum and minimum temperature variables.
 3. /toggle1 and /toggle2: change status of sensors 1 and 2.
 4. /switch_scale: change unit variable between 'C' and 'F'.
-

Algorithm 5: /toggle Endpoint

Global *sensorEnabled* **Input:** httpRequest*toggle* $\leftarrow$ **toggle** parameter from httpRequest;**if** *toggle* = "ON" **then** *sensorEnabled* $\leftarrow$;**end****else** *sensorEnabled* $\leftarrow$;**end**

Algorithm 6: GUI /temperature Route

Global *UnpluggedTemp*, *OffTemp*, *unit*

temperature $\leftarrow$ poll temperature from server using unit as parameter;

if *temperature* = *OffTemp* **or** *temperature* = *UnpluggedTemp* **then**

 | temperature $\leftarrow$ None;

end

Return temperature to JS fetch;

5.3 References

1. Arduino. (2024). LiquidCrystal Library for Arduino. Available at: <https://www.arduino.cc/reference/en/libraries/liquidcrystal/>.
2. Burton, M. (2024). Arduino Temperature Control Library. Available at: <https://github.com/milesburton/Arduino-Temperature-Control-Library>.
3. PJRC. (2024). OneWire Library for Arduino. Available at: https://www.pjrc.com/teensy/td_libs_OneWire.html.
4. Arduino. (2024). WiFi Library for ESP32. Available at: <https://www.arduino.cc/reference/en/libraries/wifi/>.
5. dvarrel. (2024). AsyncTCP GitHub Repository. Available at: <https://github.com/dvarrel/AsyncTCP>.
6. Arduino. (2024). WiFiUDP Library for ESP32. Available at: <https://www.arduino.cc/reference/en/libraries/wifi/wifiudp/>.
7. Pallets Projects. (2024). Flask Documentation (Version 3.0.x). Available at: <https://flask.palletsprojects.com/en/3.0.x/>.
8. Python Software Foundation. (2024). smtplib — SMTP protocol client. Available at: <https://docs.python.org/3/library/smtplib.html>.
9. Chart.js. (2024). Simple, flexible JavaScript charting for designers & developers. Available at: <https://www.chartjs.org/>.
10. jQuery. (2024). jQuery JavaScript Library. Available at: <https://jquery.com>.
11. Python Software Foundation. (2024). tkinter — Python interface to Tcl/Tk. Available at: <https://docs.python.org/3/library/tkinter.html>.
12. Maxim. (2008). DS18B20 Programmable Resolution 1-Wire Digital Thermometer Datasheet. Available at: <https://cdn-shop.adafruit.com/datasheets/DS18B20.pdf>.
13. Espressif Systems. (2023). ESP-32-WROOM-32 Datasheet. Available at: https://www.espressif.com/sites/default/files/documentation/esp32-wroom-32_datasheet_en.pdf.

14. Espressif Systems. (2024). ESP 32 Series Version 4.7 Datasheet. Available at: https://www.espressif.com/sites/default/files/documentation/esp32_datasheet_en.pdf.
15. Galaxy Microelectronics. A-1117 Low Dropout Regulator Datasheet. Available at: <https://pdf.dzsc.com/130319/A1117%20S0T-89.pdf>.
16. Longtech Display. LCM 1602A LCD Spec Sheet. Available at: <https://www.sigmaelectronica.net/manuals/LCM1602A.pdf>.
17. Hitachi. (1998). HD44780U LCD II Controller/Driver Datasheet. Available at: <https://www.sparkfun.com/datasheets/LCD/HD44780.pdf>.
18. Switchcraft/Conxall. (2019). SWITCHCRAFT/CONXALL W6280-3SG-P-3DC Drawing. Available at: <https://www.farnell.com/cad/2990821.pdf>.
19. Switchcraft/Conxall. (2019). SWITCHCRAFT/CONXALL W7280-3PG-P-300 Drawing. Available at: <https://www.farnell.com/cad/2990816.pdf>.